

Keebox

Data Backup and Synchronization Architecture Specification

Status: Consolidated architecture specification

Scope: Local-first persistence, asynchronous encrypted backup, retry behavior, server synchronization, restore, deletion semantics, background execution, API contracts, data models, and implementation reference code.

Current Keebox terminology: KBox, KCredentialBox, KNoteBox, KBKey, and KMKey.

This document supersedes the earlier backup/synchronization draft by reorganizing the design into explicit invariants, data models, state machines, API contracts, failure behavior, and implementation examples.

Contents

1. Purpose and System Boundary
2. Terminology and Current Naming
3. Architectural Goals and Non-Negotiable Invariants
4. High-Level Architecture
5. Local Persistence Model
6. The Transactional Sync Outbox
7. Backup Scheduling Policy
8. Synchronization Engine and State Machine
9. Retry, Backoff, and Connectivity Handling
10. Sync Granularity and Batching
11. Push Protocol and Idempotency
12. Server-Side Persistence Model
13. Revisions, Server Sequence, and Incremental Pull
14. Conflict Behavior and Future Multi-Device Readiness
15. Deletion and Tombstones
16. Restore Discovery and Restore Workflow
17. Background and Resumable Restore
18. Logout, Uninstall, Device Replacement, and Offline Edge Cases
19. KBKey/KMKey Integration with Backup and Restore
20. Security and Privacy Rules for Synchronization
21. Performance and Storage Considerations
22. User Experience and Sync Status
23. Recommended API Surface
24. Reference Local SQL Schema
25. Reference Flutter/Dart Models and Repository Flow
26. Reference Flutter Sync Worker
27. Reference Django Ninja Schemas and Endpoints
28. Reference Backend Sync Service
29. Restore Worker Reference Flow
30. Failure Matrix and Recovery Rules
31. Testing Strategy
32. Final Architecture Summary

1. Purpose and System Boundary

Keebox is an offline-first mobile application. KCredentialBox and KNoteBox data are primarily stored on the Android device in encrypted local SQLite storage. The Django Ninja backend exists as a secondary persistence layer for backup, restore, and synchronization rather than as a dependency for ordinary local use.

The central problem solved by this architecture is: **how can Keebox reliably maintain a recoverable remote copy of encrypted KBoxes without making network availability part of the save experience?**

The answer is to separate local persistence from network synchronization. Every user mutation succeeds locally first. A durable local synchronization record is created at the same time. A background sync engine later pushes the encrypted change when policy and connectivity allow it.

Out of scope for this document

- The internal field structure of KCredentialBox and KNoteBox beyond what synchronization requires.
- The detailed cryptographic implementation of KBKey and KMKey, except where key handling affects backup or restore.
- Collaborative editing or shared KBoxes between multiple users.
- Server-side searching inside encrypted KBox payloads.

2. Terminology and Current Naming

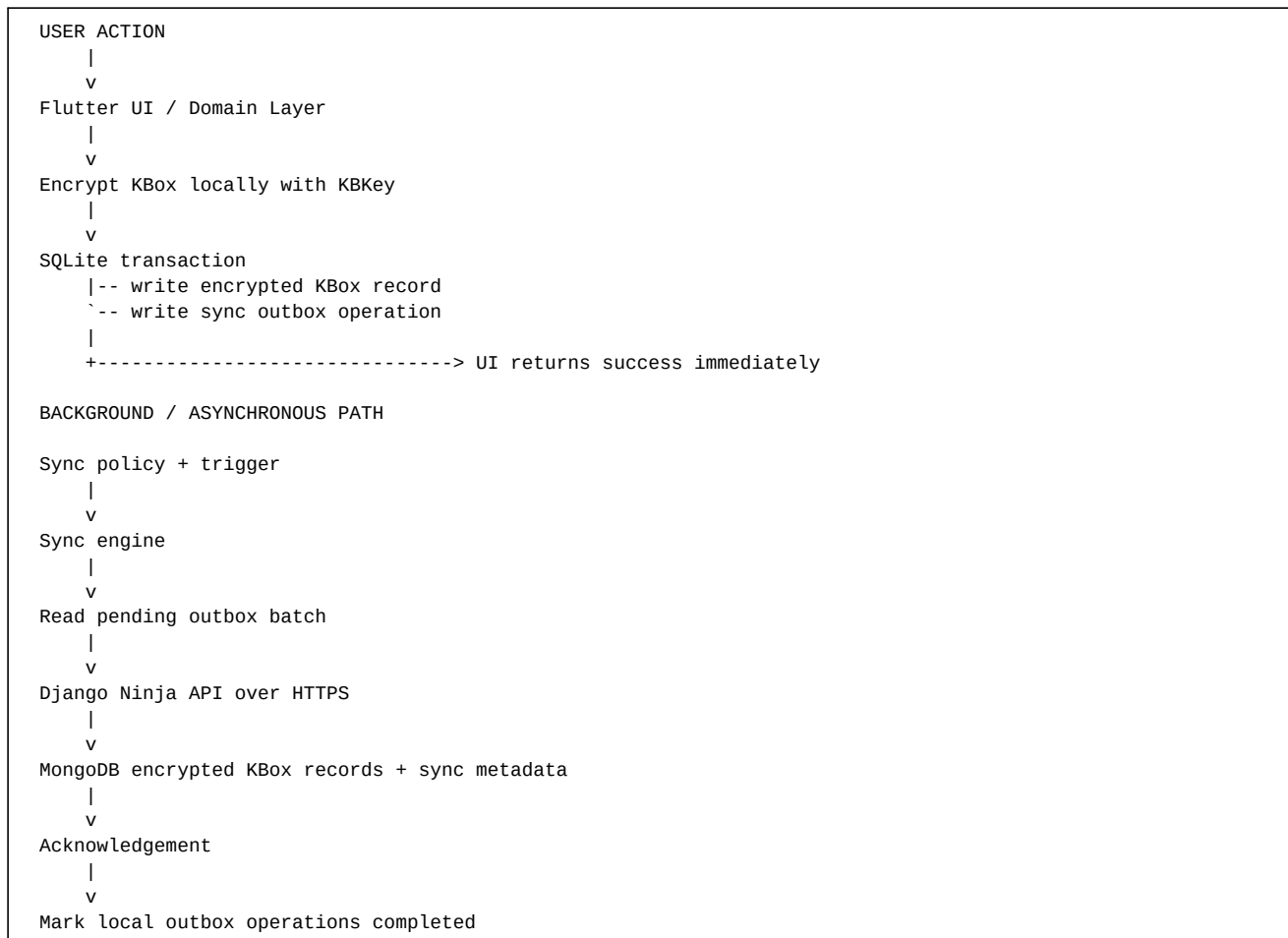
Term	Meaning in this specification
KBox	The common protected-data concept used by Keebox.
KCredentialBox	A KBox containing credential-related private data such as account secrets, passwords, API keys, or similar information.
KNoteBox	A KBox containing private text notes.
KBKey	The single stable per-user Keebox Key used by the client to encrypt and decrypt all of that user's KBoxes.
KMKey	The Keebox Master Key held by the backend and used to protect stored KBKeys.
Outbox operation	A durable local record that states that a KBox mutation still needs to be synchronized.
Server sequence	A monotonically increasing per-user sequence assigned to accepted remote changes so clients can pull incrementally.
Tombstone	A deletion marker retained long enough to prevent a deleted KBox from reappearing during synchronization or restore.

3. Architectural Goals and Non-Negotiable Invariants

- **Local save must not wait for the network.** A normal KBox create, edit, or delete operation is a local database operation first.
- **Offline operation is expected.** Lack of internet access is not a save error.
- **Every unsynchronized mutation must be durable.** The app must survive process death, phone restart, and temporary network failure without forgetting what still needs to be uploaded.
- **Data is encrypted before synchronization.** The backend receives KBox ciphertext and synchronization metadata, not raw credential or note contents.
- **Synchronization must be idempotent.** Retrying a request after a lost response must not create duplicate KBoxes.
- **Restore must be resumable.** Large backups cannot require one uninterrupted foreground session.
- **Deletion must synchronize explicitly.** A locally deleted box must not return on the next restore.

- **Key recovery must survive logout, reinstall, and password reset.** The backend can recover the user KBKey through the KMKey architecture and return it after authentication.

4. High-Level Architecture



The local SQLite database is the active working store. The remote backend is a recoverable replica of encrypted records. The synchronization engine connects the two without coupling the local user experience to network success.

5. Local Persistence Model

The local database needs at least three logical areas: encrypted KBox records, the sync outbox, and synchronization state/checkpoints.

Local entity	Purpose	Important fields
kboxes	Stores the active encrypted KCredentialBox and KNoteBox records.	id, type, ciphertext, nonce, local_revision, remote_revision, created_at, updated_at, deleted
sync_outbox	Durably records mutations that still require remote acknowledgement.	operation_id, entity_id, operation, entity_revision, status, attempt_count, next_attempt_at, created_at
sync_state	Stores device/account synchronization checkpoints.	last_server_sequence, last_successful_sync_at, restore_cursor, pending_restore, last_error
backup_policy	Stores user-selected backup timing and network constraints.	mode, weekday/day_of_month, preferred_time, wifi_only, charging_only, enabled

Encrypted KBox record envelope

```
{
  "id": "UUID",
  "type": "credential" | "note",
  "schema_version": 1,
  "ciphertext": "...",
  "nonce": "...",
  "local_revision": 12,
  "remote_revision": 11,
  "deleted": false,
  "created_at": "...",
  "updated_at": "..."
}
```

The database does not need to understand the plaintext fields inside the KBox. Its responsibility is to persist an encrypted envelope and the metadata required for local lifecycle and synchronization.

6. The Transactional Sync Outbox

The outbox is the core reliability mechanism. Every mutation that should eventually exist remotely creates an outbox operation in the same SQLite transaction as the KBox change.

```
BEGIN TRANSACTION

1. Insert/update/delete-mark encrypted KBox record
2. Increment local revision
3. Insert sync_outbox operation with a fresh operation_id

COMMIT
```

Why the same transaction matters

- If the KBox write succeeds but the outbox write fails, the backend may never learn that the KBox changed.
- If the outbox write succeeds but the KBox write fails, the sync worker may attempt to upload a state that does not exist locally.
- A single SQLite transaction guarantees that both changes become durable together or neither does.

Recommended outbox states

State	Meaning
pending	Eligible for synchronization when policy and retry timing allow.
in_flight	Currently included in a request. Must be safely recoverable to pending after a crash or timeout.
failed	Last attempt failed but the operation remains retryable.
completed	Backend acknowledged the operation. The row may be retained briefly for diagnostics or removed.

7. Backup Scheduling Policy

The architecture separates **change capture** from **when network backup is allowed to run**. Every mutation is captured immediately in the outbox. A user-selected backup policy decides when the sync engine should attempt to push the backlog.

Policy mode	Behavior
On change / automatic	Attempt soon after changes, subject to connectivity and background execution limits.
Daily	Attempt once the configured daily backup window becomes due.
Weekly	Attempt on/after the selected weekday and preferred time.
Monthly	Attempt on/after the selected day-of-month and preferred time.

Policy mode	Behavior
Manual	Only a user-initiated Sync now / Back up now action starts a new backup attempt.

Android background execution is best-effort. A preferred time should therefore be treated as a target window, not a guarantee that the operating system will execute Keebox at the exact minute. When a scheduled backup becomes due but the device is offline, the due state remains outstanding and the app retries when execution and connectivity become available.

Policy evaluation rule

```

if backup_disabled:
    do_not_push_automatically()
elif manual_trigger:
    run_sync()
elif policy_is_due and retry_window_allows:
    run_sync()
elif policy_is_on_change and pending_operations_exist:
    run_sync()

```

8. Synchronization Engine and State Machine

The sync engine is a background-capable application service. UI code may request that it run, but UI code must not own the actual upload lifecycle.

```

IDLE
|
| trigger + policy allows
v
PREPARING
|
| load due pending operations
v
PUSHING
| \
|  \__ success -> ACKNOWLEDGING -> more work? -> PUSHING
|
| ----- failure -> BACKOFF -> IDLE

After push completion, optional incremental pull can run:
PULLING -> APPLYING_REMOTE_CHANGES -> CHECKPOINT -> IDLE

```

Important execution rules

- Only one sync session per account/device should mutate sync state at a time.
- A process restart must not strand operations in `in_flight` forever; stale `in_flight` rows should be reset to pending.
- The worker must stop cleanly if authentication becomes invalid and surface an account-level sync error.
- The worker should process finite batches so it can checkpoint progress frequently.

9. Retry, Backoff, and Connectivity Handling

Network connectivity indicators are only hints. A device can report Wi-Fi connectivity while the internet, DNS, TLS path, or Keebox backend is unavailable. The result of the actual HTTP request determines success.

Recommended retry behavior

Failure class	Behavior
Timeout / connection failure	Keep operations pending; retry using exponential backoff with jitter.
HTTP 401/403	Stop the sync session; require authentication refresh or login. Do not keep hammering the endpoint.
HTTP 409 conflict	Do not silently overwrite; record conflict and run pull/reconciliation logic.
HTTP 429	Respect Retry-After when supplied; otherwise use a longer backoff.
HTTP 5xx	Treat as transient; retry later.
Validation / permanent 4xx	Mark the specific operation as requiring attention; do not endlessly retry unchanged invalid data.

Example backoff profile

```
Attempt 1 -> ~30 seconds
Attempt 2 -> ~2 minutes
Attempt 3 -> ~5 minutes
Attempt 4 -> ~15 minutes
Attempt 5 -> ~1 hour
Later attempts -> capped long delay + jitter
```

The exact schedule is a tunable implementation parameter. The invariant is that retries become less aggressive over time and remain durable across app restarts.

10. Sync Granularity and Batching

Keebox should synchronize individual encrypted KBoxes rather than uploading the entire SQLite database file. A single edited note should therefore produce one logical KNoteBox upsert, not a full-database backup.

- Lower bandwidth usage.
- Smaller retry units after failures.
- Independent deletion semantics.
- Incremental restore and pagination.
- Future multi-device synchronization without replacing the entire database.

Batching

The worker should group multiple due outbox operations into bounded requests. A starting batch size such as 50-100 operations is reasonable for testing, but it should remain configurable. Batch limits should also consider total request byte size rather than item count alone.

Do not compress ciphertext expecting meaningful gains. Encrypted data is intentionally high entropy and usually compresses poorly. If compression is ever required for large text payloads, it should happen before encryption on the client.

11. Push Protocol and Idempotency

Every outbox operation receives a globally unique operation_id, normally a UUID. The operation_id is the idempotency identity for that logical mutation attempt.

```
POST /sync/push
Authorization: Bearer <JWT>
```

```
{
  "operations": [
    {
      "operation_id": "uuid-op-1",
      "entity_id": "uuid-kbox-1",
      "entity_type": "credential",
      "operation": "upsert",
      "entity_revision": 12,
      "ciphertext": "...",
      "nonce": "...",
      "schema_version": 1
    }
  ]
}
```

Idempotency scenario

1. Client sends operation_id = X
2. Server commits X successfully
3. Network drops before response reaches client
4. Client retries operation_id = X
5. Server recognizes X as already applied
6. Server returns the same acknowledgement semantics
7. No duplicate KBox is created

The backend should retain enough idempotency history to recognize retries for the expected retry window. For long-term simplicity, the accepted operation_id may also be stored with a dedicated sync-operation record.

12. Server-Side Persistence Model

The backend database is MongoDB. The examples below describe recommended document shapes; exact Django MongoDB adapter syntax can be chosen during implementation.

KBox document

```
{
  "_id": "kbox-uuid",
  "user_id": "user-uuid",
  "type": "credential" | "note",
  "ciphertext": "...",
  "nonce": "...",
  "schema_version": 1,
  "revision": 12,
  "server_sequence": 408,
  "deleted": false,
  "deleted_at": null,
  "created_at": "...",
  "updated_at": "..."
}
```

Accepted sync operation document

```
{
  "operation_id": "uuid-op-1",
  "user_id": "user-uuid",
  "entity_id": "kbox-uuid",
  "entity_revision": 12,
  "server_sequence": 408,
  "result": "applied",
  "processed_at": "..."
}
```


Per-user sync state

```
{
  "user_id": "user-uuid",
  "head_sequence": 408,
  "last_backup_at": "...",
  "kbox_count": 237,
  "deleted_count": 4
}
```

13. Revisions, Server Sequence, and Incremental Pull

Keebox benefits from two different ordering concepts: an entity revision and a per-user server sequence.

Value	Scope	Purpose
revision	One KBox	Detects whether an incoming mutation is newer, duplicate, or stale for that specific KBox.
server_sequence	One user account	Orders accepted remote changes so a client can request everything after its last checkpoint.

Incremental pull

```
GET /sync/pull?after=401&limit=100
```

Response:

```
{
  "changes": [ ... sequence 402 through 408 ... ],
  "next_sequence": 408,
  "has_more": false
}
```

For the initial backup-oriented release, restore can download the current remote snapshot directly. The sequence mechanism is still valuable because it makes the design ready for safe incremental pull and future multi-device use.

14. Conflict Behavior and Future Multi-Device Readiness

The current product goal is primarily backup and restore, with the local device acting as the active source during normal use. Even so, the protocol should avoid designs that make later multi-device synchronization impossible.

Recommended initial rule

- Each local edit increments the KBox local revision.
- The server accepts an incoming revision when it is newer than the currently stored revision, or when the operation_id is a recognized retry.
- A stale revision is rejected as a conflict rather than silently overwriting newer remote state.
- The client can then pull the remote version and apply a defined reconciliation policy.

For a single-device backup workflow, conflicts should be rare. If true multi-device editing is added, Keebox can introduce explicit strategies such as last-write-wins, duplicate-on-conflict, or user-assisted resolution. That policy is intentionally not fixed here.

15. Deletion and Tombstones

Deleting a KBox must be synchronized as data. Simply removing the local row would leave the remote copy intact and allow the item to reappear during a future restore.

```

User deletes KBox
|
v
Local record becomes deleted/tombstoned
|
v
Delete operation inserted into outbox
|
v
Backend receives delete
|
v
Remote record becomes deleted/tombstoned
|
v
Server acknowledges

```

Two distinct destructive actions

- **Delete one KBox:** synchronizes a tombstone so that KBox remains deleted across devices/restores.
- **Delete all server backup data:** an explicit account-level action that removes the user's remote KBox backup according to the product's deletion policy.

Tombstones may eventually be garbage-collected after a configurable retention period once the system can prove that their deletion state no longer needs to propagate. No fixed retention duration is mandated by this specification.

16. Restore Discovery and Restore Workflow

After login on a fresh installation, a replacement device, or a logout flow that removed local KBox data, the client asks the backend whether remote KBoxes exist.

```

Login succeeds
|
v
Acquire authenticated session and KBKey
|
v
GET /backup/status
|
+-- no remote KBoxes -> continue to home
|
`-- remote KBoxes exist
    |
    v
    Show restore choice
    |
    +-----+-----+
    |         |         |
    | Restore |         | Skip
    |         |         |
    |         |         |
    v         |         |
Download encrypted pages -> store locally -> checkpoint progress

```

Restore rules

- Download ciphertext and metadata; do not require plaintext on the backend.
- Persist each downloaded page/batch locally before advancing the restore cursor.
- Do not create normal outbound sync operations for records being inserted as part of restore.
- Decrypt KBoxes when the UI needs them rather than requiring the entire backup to be decrypted before home becomes usable.
- When restore completes, store the server head sequence as the client's synchronization checkpoint.

17. Background and Resumable Restore

A large backup must not trap the user on a blocking progress screen. Keebox should offer a background option that returns the user to the home screen while a resumable worker continues.

Required restore checkpoint data

Field	Purpose
restore_session_id	Identifies the logical restore attempt.
next_cursor / next_page	Tells the worker where to resume.
downloaded_count	Progress reporting.
total_count	Optional server-provided estimate for UI progress.
started_at / updated_at	Lifecycle and diagnostics.
status	pending, running, paused, completed, failed.

Android background work should use a WorkManager-style mechanism so the operating system can resume network work after process death. The restore worker should be idempotent: replaying a downloaded page must upsert the same local KBox IDs rather than duplicate them.

18. Logout, Uninstall, Device Replacement, and Offline Edge Cases

Logout

A logout can safely revoke authentication tokens and clear the KBKey from active memory. A recommended privacy/reliability policy is to preserve the encrypted local database unless the user explicitly requests local-data removal. This keeps unsynchronized ciphertext available for a later login without leaving the key active.

If the product offers "logout and remove local data", Keebox should first inspect the outbox. If unsynchronized operations exist and the device cannot complete backup, the UI should clearly warn that deleting local data can permanently lose those changes.

Uninstall

The operating system removes the application's local database and device-stored key material. Only operations already acknowledged by the backend are guaranteed to be recoverable. No software architecture can remotely preserve an offline change if the user uninstalls the only device copy before it synchronizes.

Device replacement

The replacement device authenticates, receives the same KBKey through the backend key-management path, discovers the remote backup, and restores the encrypted KBoxes.

19. KBKey/KMKey Integration with Backup and Restore

Synchronization depends on the current Keebox key architecture but does not change its responsibilities. The KBKey encrypts and decrypts KBoxes on the client. The KMKey protects KBKeys on the backend.

SERVER KEY PATH

KMKey

|

| AES-256-GCM

v

encrypted KKey stored for user

RESTORE / LOGIN

encrypted KKey

|

| decrypt using KMKey on backend

v

KKey

|

| HTTPS to authenticated Flutter client

v

client decrypts restored KBoxes locally

This is deliberately not a strict zero-knowledge backend. The backend can recover a KKey because recovery after password reset, reinstall, and device replacement is a required product behavior.

- Never include KKey or KMKey in logs, analytics, crash reports, exception text, or sync payload diagnostics.
- The KKey should only be returned after successful account authentication and authorization.
- KMKey must remain outside MongoDB and be supplied through secure server secret configuration.
- All key transfer occurs over HTTPS/TLS.

20. Security and Privacy Rules for Synchronization

Rule	Requirement
Plaintext boundary	Raw KCredentialBox/KNoteBox content remains on the client during backup.
Transport	All synchronization and key-delivery endpoints require HTTPS.
Authentication	Sync/backup endpoints require a valid authenticated user context, e.g. JWT.
Ownership	Every backend lookup/upsert must scope by authenticated user, never by KBox ID alone.
Logging	Log IDs, counts, timing, and error categories only; never ciphertext if avoidable, and never keys or plaintext.
Integrity	AES-GCM authentication failures must be treated as corruption/tampering, not as recoverable plaintext.
Replay	operation_id idempotency prevents accidental duplicate application of retried requests.

21. Performance and Storage Considerations

- Keep user-facing local database transactions short. Encryption can happen immediately before the transaction; the transaction should persist the completed ciphertext plus outbox record.
- Use bounded sync batches and pagination. Avoid reading an entire large backup into memory.
- Index MongoDB by user_id + entity id, user_id + server_sequence, and operation_id according to the chosen collections.
- Use local indexes on outbox status + next_attempt_at and KBox id/type.
- Checkpoint after every successful batch so process death loses at most one in-flight batch of work.
- Avoid redundant outbox growth. Repeated unsent upserts to the same KBox may be coalesced when safe, as long as the latest encrypted state and revision remain represented.

Outbox coalescing note

A simple first version can keep every operation. A later optimization can collapse multiple pending upserts for the same entity into the newest revision before network transmission. Delete operations must not be incorrectly removed by coalescing.

22. User Experience and Sync Status

Synchronization should remain visible enough to build trust without turning normal local saves into network workflows.

State	Suggested user-facing meaning
Up to date	No pending operations and most recent eligible backup succeeded.
Waiting for internet	Pending operations exist but the device/backend is currently unreachable.
Backup scheduled	Changes are captured locally and the configured policy is not yet due.
Syncing	Background worker is currently pushing/pulling.
Backup issue	A persistent authentication, validation, or server error requires attention.
Restoring	Remote encrypted KBoxes are being downloaded; may continue in background.

A KBox save should still show success once the local transaction commits. The sync status is separate and can be shown at account/settings level or as a subtle global indicator.

23. Recommended API Surface

Method	Endpoint	Purpose
POST	/sync/push	Upload a batch of encrypted upsert/delete operations.
GET	/sync/pull	Retrieve accepted remote changes after a server sequence.
GET	/backup/status	Report whether remote backup exists, counts, size estimate, and server head sequence.
GET	/backup/restore	Page through current encrypted KBoxes for restore.
DELETE	/backup	Explicitly delete the authenticated user's remote KBox backup.
POST	/backup/sync-now	Optional server-side audit/command endpoint if the product needs a manual action record; not required for basic push.

Backup status response example

```
{
  "has_backup": true,
  "kbox_count": 237,
  "credential_count": 91,
  "note_count": 146,
  "head_sequence": 408,
  "last_backup_at": "2026-09-03T08:45:00Z"
}
```

24. Reference Local SQL Schema

The following is a reference SQLite shape. It is intentionally implementation-oriented and can be adapted to the Flutter persistence library selected for Keebox.

```

CREATE TABLE kboxes (
  id TEXT PRIMARY KEY,
  type TEXT NOT NULL CHECK (type IN ('credential', 'note')),
  schema_version INTEGER NOT NULL DEFAULT 1,
  ciphertext BLOB NOT NULL,
  nonce BLOB NOT NULL,
  local_revision INTEGER NOT NULL DEFAULT 1,
  remote_revision INTEGER NOT NULL DEFAULT 0,
  deleted INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE sync_outbox (
  operation_id TEXT PRIMARY KEY,
  entity_id TEXT NOT NULL,
  operation TEXT NOT NULL CHECK (operation IN ('upsert', 'delete')),
  entity_revision INTEGER NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending',
  attempt_count INTEGER NOT NULL DEFAULT 0,
  next_attempt_at TEXT,
  created_at TEXT NOT NULL,
  last_error_code TEXT,
  FOREIGN KEY(entity_id) REFERENCES kboxes(id)
);

CREATE INDEX idx_outbox_due
ON sync_outbox(status, next_attempt_at);

CREATE TABLE sync_state (
  account_id TEXT PRIMARY KEY,
  last_server_sequence INTEGER NOT NULL DEFAULT 0,
  last_successful_sync_at TEXT,
  restore_cursor TEXT,
  restore_status TEXT,
  restore_downloaded_count INTEGER NOT NULL DEFAULT 0,
  restore_total_count INTEGER
);

```

25. Reference Flutter/Dart Models and Repository Flow

Reference models

```

enum KBoxType { credential, note }
enum SyncOperationType { upsert, delete }
enum SyncStatus { pending, inFlight, failed, completed }

class EncryptedKBoxRecord {
  final String id;
  final KBoxType type;
  final int schemaVersion;
  final List<int> ciphertext;
  final List<int> nonce;
  final int localRevision;
  final int remoteRevision;
  final bool deleted;
  final DateTime createdAt;
  final DateTime updatedAt;
}

class SyncOutboxEntry {
  final String operationId;
  final String entityId;
  final SyncOperationType operation;
  final int entityRevision;
  final SyncStatus status;
  final int attemptCount;
  final DateTime? nextAttemptAt;
  final DateTime createdAt;
}

```

Atomic repository mutation

```
Future<void> saveEncryptedKBox(EncryptedKBoxRecord record) async {
  await database.transaction(() async {
    final nextRevision = record.localRevision + 1;

    await kboxDao.upsert(
      record.copyWith(localRevision: nextRevision),
    );

    await outboxDao.insert(
      SyncOutboxEntry(
        operationId: uuid.v4(),
        entityId: record.id,
        operation: SyncOperationType.upsert,
        entityRevision: nextRevision,
        status: SyncStatus.pending,
        attemptCount: 0,
        nextAttemptAt: null,
        createdAt: DateTime.now().toUtc(),
      ),
    );
  });

  // UI success is based on the committed local transaction.
  syncScheduler.requestOpportunisticRun();
}
```

The repository example assumes the record is already encrypted with the user KBKey. The exact database library may be Drift, sqflite, or another SQLite abstraction; the transaction semantics are the important part.

26. Reference Flutter Sync Worker

```
Future<SyncRunResult> runSync() async {
  if (!await policy.isAllowedNow()) {
    return SyncRunResult.notDue;
  }

  if (!await auth.hasUsableSession()) {
    return SyncRunResult.authenticationRequired;
  }

  while (true) {
    final batch = await outbox.loadDueBatch(limit: 100);
    if (batch.isEmpty) break;

    await outbox.markInFlight(batch);

    try {
      final request = await buildPushRequest(batch);
      final response = await api.push(request);

      await database.transaction(() async {
        await applyAcknowledgements(response);
        await outbox.completeAcknowledged(response.acknowledgedIds);
      });
    } on AuthenticationException {
      await outbox.returnToPending(batch);
      return SyncRunResult.authenticationRequired;
    } on TransientNetworkException catch (error) {
      await outbox.scheduleRetry(batch, error);
      return SyncRunResult.retryLater;
    } catch (error) {
      await outbox.failOrSchedule(batch, error);
      return SyncRunResult.partialFailure;
    }
  }

  await pullRemoteChangesIfEnabled();
  await syncState.markSuccessfulNow();
  return SyncRunResult.success;
}
```

Production code should add cancellation, stale in-flight recovery, per-operation result handling, request byte limits, structured error categories, and background-worker integration.

27. Reference Django Ninja Schemas and Endpoints

Push schemas

```
from typing import Literal
from uuid import UUID
from ninja import Schema

class SyncOperationIn(Schema):
    operation_id: UUID
    entity_id: UUID
    entity_type: Literal["credential", "note"]
    operation: Literal["upsert", "delete"]
    entity_revision: int
    schema_version: int = 1
    ciphertext: str | None = None
    nonce: str | None = None

class SyncPushIn(Schema):
    operations: list[SyncOperationIn]

class SyncOperationResult(Schema):
    operation_id: UUID
    status: Literal["applied", "duplicate", "conflict", "rejected"]
    remote_revision: int | None = None
    server_sequence: int | None = None

class SyncPushOut(Schema):
    results: list[SyncOperationResult]
    head_sequence: int
```

Endpoint skeleton

```
@router.post("/sync/push", response=SyncPushOut, auth=jwt_auth)
def push_sync_operations(request, payload: SyncPushIn):
    user = request.auth
    results = sync_service.apply_batch(
        user=user,
        operations=payload.operations,
    )
    return {
        "results": results.items,
        "head_sequence": results.head_sequence,
    }
```

Restore discovery skeleton

```
@router.get("/backup/status", auth=jwt_auth)
def backup_status(request):
    user = request.auth
    return backup_service.status_for(user)

@router.get("/backup/restore", auth=jwt_auth)
def restore_page(request, cursor: str | None = None, limit: int = 100):
    user = request.auth
    return backup_service.restore_page(
        user=user,
        cursor=cursor,
        limit=min(limit, 200),
    )
```

28. Reference Backend Sync Service

The critical backend behavior is ownership scoping, idempotency, revision checks, and atomic assignment of server sequence values. The code below is intentionally service-layer pseudocode because the exact MongoDB integration choice can vary.


```

def apply_operation(user, op):
    # 1. Idempotency check
    previous = sync_ops.find_one({
        "operation_id": str(op.operation_id),
        "user_id": str(user.id),
    })
    if previous:
        return duplicate_result(previous)

    # 2. Load only within the authenticated user's ownership scope
    existing = kboxes.find_one({
        "_id": str(op.entity_id),
        "user_id": str(user.id),
    })

    # 3. Reject stale revisions
    if existing and op.entity_revision <= existing["revision"]:
        return conflict_result(existing)

    # 4. Allocate next per-user server sequence atomically
    sequence = next_user_sequence(user.id)

    # 5. Apply upsert/delete using ciphertext only
    if op.operation == "upsert":
        validate_ciphertext_fields(op)
        upsert_remote_kbox(user, op, sequence)
    else:
        apply_tombstone(user, op, sequence)

    # 6. Persist idempotency result
    sync_ops.insert_one({
        "operation_id": str(op.operation_id),
        "user_id": str(user.id),
        "entity_id": str(op.entity_id),
        "entity_revision": op.entity_revision,
        "server_sequence": sequence,
        "result": "applied",
    })

    return applied_result(op, sequence)

```

For a production batch endpoint, transaction/session capabilities and atomic update patterns should be chosen according to the MongoDB deployment topology. The protocol semantics above remain the same even if persistence details differ.

29. Restore Worker Reference Flow

```
Future<void> resumeRestore() async {
    var checkpoint = await restoreState.load();

    while (!checkpoint.completed) {
        final page = await api.restorePage(
            cursor: checkpoint.nextCursor,
            limit: 100,
        );

        await database.transaction(() async {
            for (final remote in page.items) {
                await kboxDao.upsertFromRestore(remote);
            }

            await restoreState.save(
                nextCursor: page.nextCursor,
                downloadedCount:
                    checkpoint.downloadedCount + page.items.length,
                completed: page.nextCursor == null,
            );
        });

        checkpoint = await restoreState.load();
    }

    await syncState.setLastServerSequence(
        checkpoint.serverHeadSequence,
    );
}
```

Restored inserts should bypass normal outbox creation because they already represent acknowledged server state. If restore is interrupted, the checkpoint guarantees that the next run can continue.

30. Failure Matrix and Recovery Rules

Scenario	Expected behavior
User saves while offline	Local encrypted save succeeds; outbox remains pending; no blocking error.
App killed after local save	Outbox survives in SQLite and sync resumes later.
Backend stores push but response is lost	Same operation_id is retried; backend returns duplicate/previous acknowledgement without duplicate data.
Network returns after scheduled backup was missed	Due policy remains outstanding; worker retries when execution is available.
JWT expires mid-sync	Stop sync, preserve pending work, refresh/re-authenticate before continuing.
KBox deleted offline	Local tombstone + delete outbox operation; server deletion catches up later.
Restore interrupted	Checkpoint remains; worker resumes at saved cursor/page.
User resets login password	KBKey remains independent; authenticated backend returns same KBKey and old backup stays decryptable.
User uninstalls with unsynced offline changes	Those unsynced changes cannot be recovered because no remote copy exists.
Corrupted ciphertext/authentication failure	Do not output plaintext; mark item as corrupted and preserve diagnostics without logging secrets.

31. Testing Strategy

Unit tests

- Policy due/not-due calculation for daily, weekly, monthly, and manual modes.
- Backoff calculation and retry classification.

- Outbox state transitions and stale in-flight recovery.
- Revision comparison and idempotency behavior.
- Restore checkpoint advancement.

Integration tests

- Local KBox write + outbox write commit atomically.
- Push batch -> MongoDB persistence -> acknowledgement -> outbox completion.
- Duplicate operation_id does not create duplicate KBoxes.
- Delete tombstone prevents reappearance during restore.
- Incremental pull after a saved server sequence returns only newer changes.
- JWT/authorization prevents one user from reading or mutating another user's KBoxes.

Failure and lifecycle tests

- Kill the app immediately after local commit and confirm pending sync resumes.
- Simulate connection loss after server commit but before response delivery.
- Simulate days of offline use followed by network recovery.
- Interrupt a large restore repeatedly and verify exact resumability.
- Logout and login again with existing encrypted local data.
- Fresh reinstall restore with KBKey retrieval and encrypted backup download.

Security tests

- Verify logs never contain KBKey, KMKey, plaintext KBox content, passwords, API keys, or note text.
- Verify tampered AES-GCM ciphertext fails authentication.
- Verify all backend KBox queries are user-scoped.
- Verify KBKey-returning endpoints require authenticated ownership.

32. Final Architecture Summary

PRIMARY DATA PATH

```
graph TD
    A[KCredentialBox / KNoteBox] -->|encrypt locally with KBKey| B[Encrypted SQLite record <--- authoritative active local copy]
    B -->|same transaction| C[Sync outbox operation]
    C -->|policy + background worker + retry| D[Django Ninja / JWT]
    D --> E[MongoDB encrypted KBox record]
```

RESTORE PATH

```
graph TD
    A[Authenticated login] -->|backend recovers user's KBKey using KMKey| B[Flutter receives KBKey over HTTPS]
    B --> C[Backup status -> restore choice]
    C --> D[Paginated encrypted KBox download]
    D --> E[Encrypted SQLite persistence + restore checkpoint]
    E --> F[KBoxes decrypted locally when required]
```

Final decisions

- Keebox is local-first and offline-first.
- Normal local saves never depend on backend availability.
- Every mutation creates a durable outbox entry atomically with the local KBox change.
- Backup execution is asynchronous and governed by a configurable policy plus retry rules.
- Pending operations survive app restarts and network outages.
- Individual encrypted KBoxes are synchronized in bounded batches.
- `operation_id` provides idempotent retries.
- Entity revisions prevent stale overwrites; per-user server sequence enables incremental pull.
- Deletion is synchronized through tombstones.
- Restore is optional, paginated, resumable, and may run in the background.
- KBKey remains independent of the login password and is recoverable through the KMKey-backed server key path.
- The backend stores encrypted KBox payloads, while the client performs KBox encryption/decryption.

The resulting design preserves a fast local experience even with unreliable connectivity while still providing a durable remote recovery path after logout, reinstall, or device replacement.